

Arquitectura de la plataforma MOOC

Entrega 2: despliegue básico en Google Cloud Platform

Proyecto GCP	<code>proyecto-plataforma-mooc</code>
Periodo académico	2026-20

Resumen. La solución distribuye un backend modular en Go y sus procesos asíncronos entre dos máquinas virtuales. PostgreSQL se aloja en Cloud SQL; los archivos se conservan en Cloud Storage. Redis conecta la API con los workers, mientras una VPC con dos subredes, reglas de firewall, Cloud NAT y conexión privada a servicios administrados organiza las comunicaciones.

Alcance de la evidencia. El documento combina recursos consultados en GCP, archivos del repositorio actualizado y resultados de pruebas conservados por el equipo. Durante el inventario las VM y Cloud SQL estaban detenidos. La configuración declarada en Docker Compose no se confunde con una inspección de procesos en ejecución. Los puntos pendientes se identifican expresamente.

Índice

1. Contexto y alcance	2
2. Modelo de componentes	2
2.1. Responsabilidades y organización del backend	2
2.2. Persistencia y flujo multimedia	3
3. Modelo de despliegue en GCP	4
3.1. Inventario de recursos	4
3.2. Topología y límites de red	5
3.3. Contenedores y volúmenes	5
3.4. Firewall, acceso y estado de la revisión	5
4. Decisiones y adaptaciones	6
4.1. Separación de responsabilidades	6
4.2. Migración de MinIO local a Cloud Storage	6
4.3. Identidad, permisos y puntos por completar	6
5. Operación y recuperación	8
5.1. Procedimiento de despliegue reproducible	8
5.2. Migración y comprobaciones	9
5.3. Reinicio y recuperación	9
6. Capacidad, costos y limitaciones	11
6.1. Resultados disponibles y alcance	11
6.2. Estimación de costos	11
6.3. Puntos únicos de falla y evolución	13
6.4. Estado de los aspectos no acreditados	13
A. Fuentes y trazabilidad	14

1 Contexto y alcance

La entrega migra la plataforma de cursos masivos de un entorno de contenedores locales a infraestructura como servicio. Conserva el monolito modular, los workers independientes, PostgreSQL, Redis/Asynq y el almacenamiento de objetos. El alcance no requiere frontend nuevo, autoscaling, balanceadores, réplicas de aplicación entre máquinas, CDN ni alta disponibilidad.

El objetivo de este documento es explicar la arquitectura, sus decisiones, la operación y recuperación, y sus límites de capacidad y costo. Se emplean tres fuentes con alcances diferentes:

- **Observado en GCP:** metadatos de VM, discos, red, Cloud SQL, buckets y respaldos, registrados en `docs/entrega2/evidencias/inventario-gcp.json`.
- **Declarado en el código:** implementación y Compose del commit `d3d6125`. Acreditan cómo está preparada la aplicación, pero no el digest exacto instalado en cada VM.
- **Medido por el equipo:** resultados JSON y métricas de las corridas referenciadas en `capacity-planning/pruebas_de_carga_entrega2.md`. No se ejecutaron nuevamente durante esta revisión.

Esta separación permite documentar una infraestructura sencilla sin atribuirle controles o resultados que no estén sustentados. La consulta cloud fue de solo lectura: no se encendieron servicios, modificaron permisos ni descargaron secretos.

2 Modelo de componentes

2.1 Responsabilidades y organización del backend

El backend usa Go 1.25 y el router Chi. Los módulos organizan handlers HTTP, casos de uso y repositorios PostgreSQL. La API se inicia con `cmd/api`; el proceso asíncrono, con `cmd/worker`. Ambos binarios se construyen con el mismo Dockerfile, que incorpora FFmpeg en la imagen de ejecución.

Componente	Responsabilidad	Dependencias
Usuarios	Registro, verificación, sesiones, roles y auditoría.	PostgreSQL; sesiones con tokens opacos persistidos.
Cursos	Cursos, versiones, módulos, unidades, recursos y publicación.	PostgreSQL y adaptador de objetos.
Aprendizaje	Inscripciones, quizzes, intentos, progreso e insignias verificables.	PostgreSQL y dominios de usuarios/cursos.
Multimedia	URLs firmadas, confirmación de cargas y entrega de manifiestos HLS.	Objetos, PostgreSQL y Asynq.
Worker	Transcodificación de audio/video y actualización de estados.	Redis, FFmpeg, disco temporal, objetos y PostgreSQL.
Redis	Cola Asynq, idempotencia HTTP y límites de peticiones.	Volumen persistente con AOF en Compose cloud.

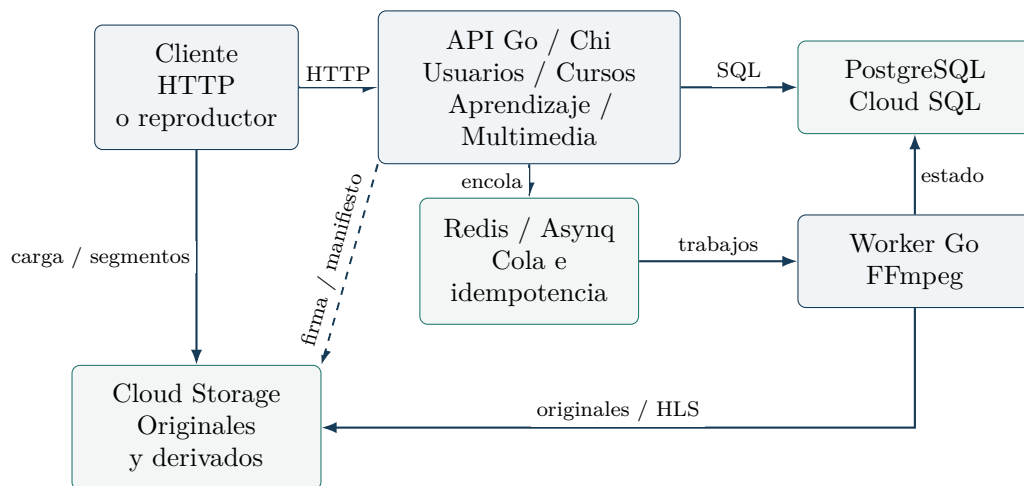


Figura 1: Modelo lógico de componentes del código actualizado. Las conexiones SQL, HTTP y objetos son síncronas; Redis desacopla la producción y ejecución de trabajos.

2.2 Persistencia y flujo multimedia

PostgreSQL almacena identidad, estructura académica, inscripciones, intentos, calificaciones, progreso y metadatos de archivos. Los bytes de multimedia se guardan en objetos; la base conserva sus referencias. La API configura un pool de 25 conexiones abiertas, 5 inactivas y vida máxima de 15 minutos.

El flujo principal es:

1. La API genera una URL PUT firmada, válida por 24 horas. El cliente carga directamente al almacenamiento, sin transferir el archivo a través de la API.
2. El cliente confirma la carga. La API comprueba que exista el objeto, actualiza el recurso y encola su procesamiento con Asynq.
3. El worker descarga el original a disco temporal, ejecuta FFmpeg y sube la playlist y segmentos a `hls/<resourceID>/`. Después actualiza PostgreSQL.
4. Para HLS, la API entrega un manifiesto que reemplaza referencias relativas por URLs de descarga firmadas de dos horas. El cliente obtiene los segmentos directamente de Cloud Storage. La API sirve el manifiesto, no los segmentos.

El worker declara concurrencia 5. El procesamiento genera una sola representación H.264/AAC; no implementa una escalera de múltiples calidades. Los mecanismos de reintento e identificación de tareas reducen duplicados, pero SQL, Redis y objetos no forman una transacción distribuida. Una playlist existente tampoco demuestra que todos sus segmentos se hayan publicado correctamente.

3 Modelo de despliegue en GCP

3.1 Inventario de recursos

Recurso	Configuración observada
Proyecto y región	<code>proyecto-plataforma-mooc</code> , región <code>us-central1</code> .
Web Server	<code>web-server</code> , zona <code>us-central1-a</code> , tipo <code>e2-highcpu-2</code> : 2 vCPU y 2 GiB RAM. IP privada <code>10.0.1.2</code> ; IP externa reservada <code>35.209.105.199</code> .
Worker Server	<code>worker-server</code> , misma zona y tipo: 2 vCPU y 2 GiB RAM. IP privada <code>10.0.2.2</code> ; sin IP externa.
Discos y sistema	Disco de arranque <code>pd-balanced</code> de 30 GiB por VM (campo <code>sizeGb=30</code>); imagen Debian 12. Ambos discos tienen <code>autoDelete=true</code> .
VPC	<code>mooc-vpc</code> , personalizada, enrutamiento regional.
Subred web	<code>mooc-public-subnet</code> , <code>10.0.1.0/24</code> .
Subred worker	<code>mooc-private-subnet</code> , <code>10.0.2.0/24</code> .
Salida a Internet	<code>mooc-nat-router</code> , con <code>mooc-cloud-nat</code> , asignación automática de IP y cobertura de las subredes.
Conexión privada SQL	Peering <code>servicenetworking-googleapis-com</code> activo; rango reservado <code>10.0.3.0/24</code> .
Cloud SQL	<code>mooc-db-instance</code> , PostgreSQL 16 Enterprise, <code>db-custom-2-8192</code> : 2 vCPU/8 GiB; SSD de 20 GiB, crecimiento automático; disponibilidad ZONAL en <code>us-central1-c</code> .
Acceso SQL	IP privada <code>10.0.3.3</code> , sin IPv4 pública. TLS configurado como <code>ENCRYPTED_ONLY</code> .
Objetos multimedia	<code>mooc-media-equipo-3</code> , regional <code>US-CENTRAL1</code> , clase <code>STANDARD</code> ; prefijos observados <code>resources/</code> y <code>hls/</code> .
Objetos de insignias	<code>mooc-badges-equipo-3</code> , misma región y clase. Su existencia no prueba generación de imágenes de insignias desde el backend.

La capacidad de las VM corresponde a los 2 vCPU y 2 GiB solicitados por el enunciado. La base de datos tiene una configuración independiente, de mayor RAM. Cloud SQL se ubica en una zona distinta de las VM, pero sigue siendo monozona.

3.2 Topología y límites de red

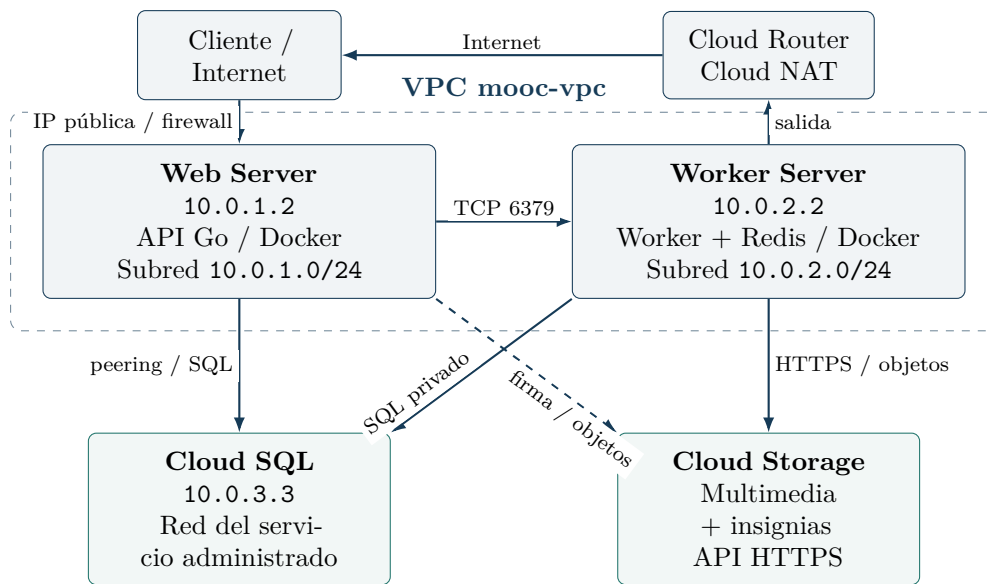


Figura 2: Recursos observados y distribución de aplicación declarada en los Compose. Las flechas representan comunicaciones previstas, no pruebas de conectividad. Cloud SQL no pertenece a la subred del worker.

La VM web tiene una IP externa reservada; el worker obtiene salida mediante NAT sin una IP pública propia. Cloud Storage es un servicio administrado accesible por HTTPS, no un contenedor ni una subred de la VPC. Las subredes registran `privateIpGoogleAccess=false`; la conectividad saliente del worker depende de la configuración de salida existente.

3.3 Contenedores y volúmenes

`deploy/docker-compose.web.yml` define un contenedor `api`, llamado `mooc-api-server`, con publicación 80:8080 y política `restart:always`. No declara proxy inverso ni terminación TLS.

`deploy/docker-compose.worker.yml` define `mooc-worker-server` y `mooc-redis`. Redis 7 publica 6379:6379, activa AOF y utiliza el volumen `redis_data:/data`; el worker espera su healthcheck. Los dos servicios también tienen reinicio automático. La publicación de Redis debe entenderse junto con el firewall privado, no como autorización pública.

3.4 Firewall, acceso y estado de la revisión

Regla	Origen y destino	Puertos
<code>allow-web-ingress</code>	Cualquier IPv4 hacia tag <code>web-server</code> .	TCP 80, 443 y 8080.
<code>allow-redis-web-to-worker</code>	10.0.1.0/24 hacia tag <code>worker-server</code> .	TCP 6379.
<code>allow-ssh-iap</code>	Rangos 35.235.240.0/20 y 0.0.0.0/0, sin tag de destino.	TCP 22.

Las reglas son configuración observada, no prueba de puertos escuchando. El nombre de la regla SSH no significa acceso exclusivo por IAP, porque también admite cualquier origen IPv4. Debe revisarse la necesidad de exponer 8080 y restringir la administración según el mecanismo elegido.

Las VM tenían estado **TERMINATED**, que en Compute Engine significa detenidas. Cloud SQL mostraba **activationPolicy=NEVER**; la consulta de bases respondió que no estaba en ejecución, aunque su campo **state** indicaba **RUNNABLE**. Por ello no se comprobó conectividad ni configuración interna del sistema operativo durante el inventario.

4 Decisiones y adaptaciones

4.1 Separación de responsabilidades

La API atiende operaciones HTTP y delega transcodificación en una cola. Separar los workers evita que FFmpeg compita directamente con las peticiones por la RAM y CPU de la VM web. Redis reside junto al worker en lugar de un servicio administrado, siguiendo el alcance básico de la entrega. La separación es de despliegue; no convierte los dominios del monolito en microservicios.

Cloud SQL administra la persistencia relacional y sus respaldos. Cloud Storage separa los archivos del ciclo de vida de los discos de las VM. Esta decisión permite conservar originales y derivados cuando un contenedor se reemplaza y evita transportar cargas o segmentos pesados por la API.

4.2 Migración de MinIO local a Cloud Storage

El código actualizado conserva **minio-go** como cliente compatible con S3. La adaptación usa el endpoint **storage.googleapis.com**, TLS y credenciales HMAC para la API XML interoperable de Cloud Storage [1]. Los nombres de variables **MINIO_*** corresponden al adaptador; no implican que se ejecute MinIO dentro de GCP.

La configuración admite **S3_REGION**, **S3_MEDIA_BUCKET** y **S3_BADGE_BUCKET**. Por tanto, el código actual permite los nombres reales con sufijo **-equipo-3**; las plantillas todavía contienen ejemplos que se deben sustituir. Los prefijos vistos en GCP son coherentes con originales y HLS, pero no constituyen por sí solos una validación de integridad ni de referencias SQL.

El flujo usa PUT firmado simple. No se acredita una carga multipart/reanudable. Los manifiestos son reescritos por la API para firmar sus segmentos, y no hay CDN en el alcance de esta entrega.

4.3 Identidad, permisos y puntos por completar

Las sesiones del backend son tokens opacos almacenados en PostgreSQL; la variable **JWT_SECRET** no demuestra uso de JWT. El código conserva accesos de desarrollo que deben revisarse antes de declarar controles de autorización completos. Las URLs firmadas limitan temporalmente el acceso a objetos, pero no sustituyen la validación de matrícula, propiedad del recurso o rol.

Ambos buckets tienen acceso uniforme y soft delete de siete días. Su prevención de acceso público figura como **inherited**; su efecto depende de la política superior, no puede deducirse privacidad absoluta de ese valor [2]. La cuenta de revisión no pudo leer IAM de buckets; Cloud Resource Manager API estaba deshabilitada y no permitió consultar IAM del proyecto. Ambas VM muestran la misma cuenta de servicio asociada. No se ha acreditado diferenciación efectiva de permisos entre API y worker.

Requisito / adaptación	Estado y acción pendiente
HTTPS público	El Compose expone HTTP 80; no hay proxy/certificado declarado. Configurar o aportar la configuración TLS real antes de afirmar cumplimiento.
TLS hacia SQL	Cloud SQL exige cifrado; las plantillas usan <code>sslmode=disable</code> . Adaptar conexión TLS o conector; no usar ese ejemplo literalmente.
Secretos	Los Compose leen archivos externos de entorno. Protegerlos, excluirlos de Git y no incluirlos en imágenes ni evidencias. La ubicación real requiere confirmación.
Permisos por componente	Documentar identidad HMAC e IAM efectivo sin registrar valores secretos; actualmente no verificado.
Procesamiento	HLS de una calidad; el handler llamado antimalware no analiza el contenido. No presentar capacidades no implementadas.

5 Operación y recuperación

5.1 Procedimiento de despliegue reproducible

Los pasos siguientes se derivan de los Compose y del inventario. Constituyen una guía de operación; no se ejecutaron durante la inspección de solo lectura.

1. Aprovisionar los recursos de la sección 3: VPC/subredes, conexión privada, Cloud SQL, buckets, VM/discos, NAT y firewall. Registrar proyecto, región, IP, tipo de máquina y política de respaldo. Se puede usar consola o infraestructura como código; el repositorio no contiene una automatización integral de estos recursos.
2. Instalar Docker Engine y Compose en ambas VM y obtener la misma revisión del repositorio. Registrar el commit y los identificadores de imagen utilizados.
3. Preparar `deploy/web-server.env` en Web Server y `deploy/worker-server.env` en Worker Server. Son los nombres exactos que referencian los Compose, no un archivo genérico `.env`.
4. Ajustar la configuración según la tabla siguiente e inyectar los secretos fuera del repositorio. La contraseña de SQL debe codificarse correctamente si se incorpora a una URL. No registrar la URL completa en evidencias.
5. Iniciar Cloud SQL y verificar su disponibilidad privada. Iniciar Redis; levantar la API para aplicar migraciones; iniciar después el worker sobre el esquema preparado. Comprobar los flujos funcionales antes de generar carga.

Variable	Web Server	Worker Server
DATABASE_URL	PostgreSQL en 10.0.3.3:5432, base/usuario autorizados y TLS. Credenciales omitidas.	
REDIS_ADDR	10.0.2.2:6379	redis:6379
MINIO_ENDPOINT	storage.googleapis.com	
MINIO_SECURE	true	true
EXTERNAL_MINIO_ENDPOINT	Vacío para usar el endpoint GCS	Vacío
S3_REGION	us-central1	us-central1
S3_MEDIA_BUCKET	mooc-media-equipo-3	
S3_BADGE_BUCKET	mooc-badges-equipo-3	
MINIO_ACCESS_KEY, MINIO_SECRET_KEY	Credenciales HMAC protegidas; permisos limitados según las operaciones necesarias.	

Para conexión directa, `sslmode=require` solicita cifrado, pero no equivale a una validación completa de identidad. Configurar validación de CA/servidor o un conector conforme al modo de certificados de Cloud SQL [3]. No se ha verificado cuál de estas alternativas utiliza el entorno desplegado.

Preparación de archivos en la VM correspondiente:

```
# Web Server, desde la raíz del repositorio
umask 077
cp -n deploy/web-server.env.example deploy/web-server.env
chmod 600 deploy/web-server.env

# Worker Server, desde la raíz del repositorio
umask 077
cp -n deploy/worker-server.env.example deploy/worker-server.env
chmod 600 deploy/worker-server.env
```

Antes de cualquier commit, excluir los dos archivos reales mediante reglas locales de Git o almacenarlos con un mecanismo externo compatible con Compose. Las reglas actuales `.env` y `.env.local` no cubren automáticamente `web-server.env` y `worker-server.env`.

Validación y arranque, una vez completada la configuración:

```
# Worker Server: primero Redis
docker compose -f deploy/docker-compose.worker.yml config --quiet
docker compose -f deploy/docker-compose.worker.yml up -d redis

# Web Server: inicializa esquema y API
docker compose -f deploy/docker-compose.web.yml config --quiet
docker compose -f deploy/docker-compose.web.yml up -d --build api

# Worker Server: inicia el consumidor
docker compose -f deploy/docker-compose.worker.yml up -d --build worker
```

El build local requiere recursos de CPU/memoria; no se ha medido su viabilidad en las VM de 2 GiB. Se pueden construir imágenes previamente, registrar sus digests y adaptar el despliegue, documentando ese cambio.

5.2 Migración y comprobaciones

La API ejecuta los archivos `backend/migrations/*.up.sql` al arrancar. No hay historial formal de migraciones ni se debe asumir un rollback automático. Para trasladar datos existentes se requiere respaldo/exportación y restauración controlada, conservando claves y referencias a objetos.

La migración de archivos debe preservar las claves de originales y derivados; comparar cantidad, tamaño y checksum disponible en origen/destino y verificar las referencias SQL. No basta con que existan los buckets. No se adjunta una bitácora de esa migración porque no está disponible entre las evidencias revisadas.

Después del arranque:

```
docker compose -f deploy/docker-compose.web.yml ps
docker compose -f deploy/docker-compose.worker.yml ps
# Desde Web Server; prueba de vida, no de todas las dependencias
curl --fail http://127.0.0.1/health
```

Ejecutar cada comando Compose en su VM. Complementar la prueba de vida con autenticación, consulta/publicación de curso, inscripción, quiz y reenvío duplicado, progreso, carga firmada, procesamiento y descarga de segmentos. Registrar códigos, cambios de estado y un fallo con reintento. Confirmar HTTPS externo cuando se haya configurado su terminación; no inferirlo de una regla 443.

5.3 Reinicio y recuperación

Para reiniciar un proceso de aplicación sin eliminar sus volúmenes:

```
# Ejecutar en la VM correspondiente
docker compose -f deploy/docker-compose.web.yml restart api
docker compose -f deploy/docker-compose.worker.yml restart worker
```

La política de reinicio de contenedores no constituye alta disponibilidad ni un respaldo. Evitar eliminar `redis_data` durante tareas rutinarias; perder la cola puede dejar recursos SQL en procesamiento sin un trabajo recuperable.

Cloud SQL tiene respaldos automáticos, retención configurada de siete copias y PITR con siete días de logs. Se observó una copia `SUCCESSFUL` del 22 de septiembre de 2026. No se probó restauración ni

se atribuye un RPO/RTO garantizado al proyecto. Cloud Storage tiene borrado recuperable de siete días, que no equivale a un respaldo independiente de la aplicación.

Procedimiento propuesto de reconstrucción:

1. Conservar commit, configuración no secreta, inventario y datos sintéticos o exportación autorizada. Guardar secretos en un medio protegido externo.
2. Recrear red, VM y Cloud SQL con las capacidades documentadas. Restaurar una copia de SQL en una instancia de recuperación antes de sustituir datos existentes, siguiendo el procedimiento del proveedor [4].
3. Recuperar objetos y sus claves; comprobar consistencia con los registros restaurados. Recuperar Redis/AOF o reconciliar y reencolar trabajos pendientes.
4. Desplegar las imágenes del commit elegido y repetir la verificación funcional. Medir tiempo real de recuperación y pérdida de datos, si existe.

Antes de eliminar la base de datos del laboratorio, como pide el enunciado, conservar exportación o datos sintéticos y una ruta de reconstrucción comprobada. Los discos de VM están asociados a borrado automático al eliminar la instancia; esta propiedad debe considerarse antes de cualquier eliminación. No se realizó ninguna acción destructiva en esta revisión.

6 Capacidad, costos y limitaciones

6.1 Resultados disponibles y alcance

Las pruebas conservadas usan k6 desde un equipo externo a las dos VM, con sesiones preparadas antes del recorrido académico. El informe declara el commit `870f259`; no debe confundirse con el commit revisado para esta documentación ni con una certificación del estado actual de las VM.

Corrida	Carga	p95 (ms)	req/s	Error
Académica, nivel 2	150 VU	255,45	56,03	0 %
Académica, repetición	400 VU	60 000,18	14,77	8,71 %

Son métricas globales HTTP. El máximo académico sin errores comprobado en esa mezcla y duración es 150 VU; la degradación aparece antes o al llegar a 400 VU, sin localizar su umbral exacto. El aumento de conexiones y las latencias sugieren investigar esperas de pool, bloqueos e I/O; no demuestran por sí solos que el pool sea la causa. No se capturó p99 académico.

En la repetición multimedia se probaron 10 VU de carga y 300 de consumo: 228,00 peticiones/s, 0 % de errores HTTP y procesamiento p95 de 6 347 ms (p99 de 9 312 ms). Se reutilizó un video de unos 90 KB y 5 segundos; no representa tres perfiles distintos ni videos educativos largos. No se midió reproducción con un reproductor ni la profundidad/antigüedad de la cola directamente. Este resultado es un máximo probado para esos datos, no la capacidad máxima del sistema.

Las fuentes exactas están en el apéndice. Los escenarios de capacidad se documentan por separado; estas limitaciones deben conservarse al interpretar sus implicaciones arquitectónicas.

6.2 Estimación de costos

Se presenta un **escenario presupuestario ilustrativo**, no el gasto real del equipo. Tarifas oficiales consultadas el 27 de septiembre de 2026 para `us-central1`, en USD bajo demanda, antes de impuestos, descuentos, créditos y Free Tier [5, 6, 7, 8, 9, 10].

Supuestos: 40 horas activas por VM y para Cloud SQL; 730 horas de conservación de discos, objetos e IP (convención mensual); 10 GiB promedio de objetos entre los dos buckets; una VM worker asignada a NAT durante 40 horas y una única IP NAT retenida 730 horas. La VM web usa su propia IPv4. Se suponen buckets de namespace plano, 1 000 operaciones clase A, 10 000 clase B, 1 GiB de salida desde GCS hacia usuarios en Colombia y otro GiB de descargas entrantes del worker procesado por NAT. Son volúmenes asumidos, no mediciones.

Concepto	Cálculo USD	Subtotal
Dos VM	$2 \times 40 \times 0,04947024$	3,95762
Discos VM, 60 GiB	$60 \times 730 \times 0,000136986$	5,99999
SQL, 2 vCPU	$2 \times 40 \times 0,0413$	3,30400
SQL, 8 GiB RAM	$8 \times 40 \times 0,007$	2,24000
SQL, SSD 20 GiB	$20 \times 730 \times 0,000232877$	3,40000
GCS Standard, 10 GiB	$10 \times 730 \times 0,000027397$	0,20000
NAT, asignación worker	$1 \times 40 \times 0,0014$	0,05600
IPv4 de NAT	$730 \times 0,005$	3,65000
IPv4 estática web	$730 \times 0,005$	3,65000
Operaciones GCS A/B	$1 \times 0,005 + 10 \times 0,0004$	0,00900
GCS hacia Colombia	$1 \text{ GiB} \times 0,12$	0,12000
Procesamiento NAT	$1 \text{ GiB} \times 0,045$	0,04500
Subtotal del escenario	Sin respaldo de tamaño desconocido	26,63

La entrega directa de GCS al usuario no pasa por Cloud NAT; el GiB de NAT del ejemplo es tráfico independiente para no contarlo dos veces. Cualquier otro egreso de VM, tráfico interzonal o volumen adicional se debe sumar. La cantidad y duración de IP automáticas NAT también pueden cambiar respecto al supuesto.

Para el respaldo SQL, cuyo tamaño facturable no fue consultado, añadir $C_{\text{backup}} = B_{\text{GiB}} \times 730 \times 0,000109589$. No se incluyen snapshots ni logs/monitorización cobrables. Las operaciones y transferencias asumidas pueden quedar cubiertas parcialmente por Free Tier; el subtotal no descuenta esa posibilidad.

Detener cómputo no elimina discos, respaldos ni objetos. La IPv4 estática web asociada sigue considerándose en uso con la VM detenida; desasociarla y mantenerla reservada tiene otra tarifa. La detención de Cloud SQL tampoco elimina sus cargos de almacenamiento [10, 11].

Contraste con consumo observado: pendiente. No se dispone de exportación de facturación ni del estado de presupuestos/alertas. Para cerrarlo, obtener el costo por servicio y periodo de las pruebas, registrar moneda/créditos, comparar horas y volúmenes con los supuestos y explicar la diferencia. Activar presupuesto y alertas si los permisos del laboratorio lo permiten; si no, registrar esa limitación. No se atribuye al equipo el plan de uso de 40 horas.

6.3 Puntos únicos de falla y evolución

Componente	Impacto de falla	Evolución posible
VM web	API y manifiestos HLS dejan de responder.	Réplicas y balanceo, después de medir el cuello de botella.
Worker/Redis	Procesamiento se detiene; cola, idempotencia y límites de API pueden afectarse.	Más workers, observabilidad y recuperación de cola.
Cloud SQL zonal	Operaciones transaccionales dejan de estar disponibles.	HA y restauración probada; revisar consultas, bloqueos y pool.
Almacenamiento y permisos	Fallan cargas, acceso a originales o segmentos.	Permisos mínimos, consistencia, alertas y futura CDN.
Zona de las VM	Ambas VM están en <code>us-central1-a</code> .	Distribución por zonas en una etapa de mayor disponibilidad.

La entrega básica no exige ejecutar estas ampliaciones. Primero conviene medir esperas SQL, memoria/disco del worker y comportamiento de la cola con archivos representativos. Más capacidad de cómputo no resuelve automáticamente un problema de bloqueo, permisos o publicación parcial de objetos.

6.4 Estado de los aspectos no acreditados

- HTTPS/proxy público y autorización completa de aplicación: no acreditados por los Compose; requieren configuración o evidencia adicional.
- IAM diferenciado API/worker y privacidad efectiva de buckets: consulta limitada por permisos de revisión y API de proyecto deshabilitada.
- Commit/digest actualmente instalado, restauración y RPO/RTO: no comprobados dentro de las VM. Los procedimientos anteriores son reproducibles como guía, no una bitácora de ejecución exitosa.
- Costos reales, horas previstas por el equipo y alertas presupuestarias: sin evidencia de facturación. Completar antes de presentar el contraste económico.

A Fuentes y trazabilidad

Tema	Archivo fuente del repositorio
Inventario GCP	docs/entrega2/evidencias/inventario-gcp.json
Consulta reproducible	docs/entrega2/scripts/inventario_gcp.py
API y worker	backend/cmd/api/main.go; backend/cmd/worker/main.go
Configuración GCS	backend/internal/config/config.go; backend/internal/media/s3.go
Manifiestos y procesamiento	backend/internal/media/manifest.go; backend/internal/worker/processor.go
Despliegue por VM	deploy/docker-compose.web.yml; deploy/docker-compose.worker.yml
Plantillas sin secretos	deploy/web-server.env.example; deploy/worker-server.env.example
Reporte de capacidad	capacity-planning/pruebas_de_carga_entrega2.md

Resultados utilizados para las cifras de capacidad:

- loadtests/results/escenario1/20260925_131343_02_nivel2_vus150.json.
- loadtests/results/escenario1/20260925_131343_04_repeticion_cerca_del_limite_vus400.json.
- loadtests/results/escenario2/20260925_141917_04_repeticion_cerca_del_limite_up10_pb300.json.

El inventario contiene 22 consultas: 17 exitosas y 5 limitadas. Se registran errores, no se interpretan como inexistencia de recursos. Se excluyeron valores de llaves, tokens, variables de contenedores y contenido de objetos. Para repetirlo, usar una cuenta de lectura y una llave fuera del repositorio:

```
python docs/entrega2/scripts/inventario_gcp.py \
  --key /ruta/prottegida/cuenta-lectura.json \
  --out /tmp/inventario-gcp.json
```

Dependencias del script: `google-auth` y `requests`. Las figuras de este documento están implementadas en TikZ dentro del propio archivo LaTeX: no requieren imágenes externas. Puede compilarse con pdfLaTeX en Overleaf.

Referencias

- [1] Google Cloud. *Interoperability with other storage providers*. <https://docs.cloud.google.com/storage/docs/interoperability>.
- [2] Google Cloud. *Public access prevention*. <https://docs.cloud.google.com/storage/docs/public-access-prevention>.
- [3] Google Cloud. *Configure SSL/TLS certificates*. <https://docs.cloud.google.com/sql/docs/postgres/configure-ssl-instance>.
- [4] Google Cloud. *Restore an instance overview*. <https://docs.cloud.google.com/sql/docs/postgres/backup-recovery/restore>.

- [5] Google Cloud. *General-purpose machine family pricing*. <https://cloud.google.com/products/compute/pricing/general-purpose>.
- [6] Google Cloud. *Disk and image pricing*. <https://cloud.google.com/compute/disks-image-pricing>.
- [7] Google Cloud. *Cloud SQL pricing*. <https://cloud.google.com/sql/pricing>.
- [8] Google Cloud. *Cloud Storage pricing*. <https://cloud.google.com/storage/pricing>.
- [9] Google Cloud. *Cloud NAT pricing*. <https://cloud.google.com/nat/pricing>.
- [10] Google Cloud. *VPC network pricing: external IP addresses*. <https://cloud.google.com/vpc/network-pricing#ipaddress>.
- [11] Google Cloud. *Start, stop, and restart instances*. <https://docs.cloud.google.com/sql/docs/postgres/start-stop-restart-instance>.